ข้อสอบปฏิบัติการ ครั้งที่ 2:

Enterprise Flutter Application Development

ระยะเวลาทำข้อสอบ: 2 สัปดาห์ (หมดเขตส่ง 16 มีนาคม 2569)

คำชี้แจง: ให้นักศึกษาพัฒนาแอปพลิเคชันด้วย Flutter โดยสามารถเลือกหัวข้อที่สนใจจาก "ตัวอย่างโปรเจกต์" ที่กำหนดให้ หรือ "นำเสนอไอเดียแอปพลิเคชันของตนเอง" ก็ได้ แต่แอปพลิเคชันนั้นจะต้องมีฟีเจอร์และโครงสร้างทางเทคนิคครบถ้วนตาม "**เกณฑ์บังคับทางเทคนิค (Core Technical Requirements)**" ด้านล่างนี้

📌 ส่วนที่ 1: เกณฑ์บังคับทางเทคนิค (ต้องมีครบทุกข้อ ไม่ว่าจะทำแอปอะไร)

1. **Architecture & State Management:**
    - ใช้โครงสร้าง Clean Architecture หรือ MVC/MVVM อย่างชัดเจน พร้อมทำ Dependency Injection (เช่น `get_it`)
    - จัดการ State ด้วย BLoC หรือ Riverpod
2. **Offline-First & Local Storage:**
    - ใช้ฐานข้อมูลหลักในเครื่อง (SQLite/sqflite, Isar, หรือ Drift) เพื่อให้แอปทำงานได้โดยไม่ต้องมีเน็ต
    - ใช้ Key-Value Storage (SharedPreferences หรือ Hive) สำหรับเก็บ Settings หรือทำ Caching
3. **Networking & API:**
    - มีการเชื่อมต่อ REST API อย่างน้อย 1 จุด โดยใช้แพ็กเกจ `dio` พร้อมจัดการ Interceptors และ Error Handling
    - ใช้ `json_serializable` หรือ `freezed` สำหรับแปลงข้อมูล JSON
4. **AI & Machine Learning Integration:**
    - **On-device ML:** มีการเรียกใช้ Google ML Kit (เช่น อ่านข้อความจากภาพ สแกนบาร์โค้ด หรือตรวจจับใบหน้า)
    - **Cloud LLM:** มีฟีเจอร์ที่เรียกใช้ LLM API (เช่น Gemini หรือ OpenAI) เพื่อประมวลผลข้อมูล (เช่น สรุปข้อความ, แนะนำแท็ก, แปลภาษา)

5. **UI, UX & Routing:**
    - ใช้ `auto_route` ในการเปลี่ยนหน้าและส่งพารามิเตอร์
    - มีหน้าจอ Form พร้อมระบบ Validation (ใช้ `GlobalKey<FormState>`)
    - มี Animation อย่างน้อย 2 จุด (Implicit 1 จุด และ Explicit/Hero 1 จุด)
6. **Testing:**
    - มี Unit Test (ใช้ Mocking), Widget Test (ทดสอบ Form) และ Integration Test ขั้นพื้นฐาน

📌 ส่วนที่ 2: ตัวอย่างหัวข้อแอปพลิเคชัน

ตัวอย่างที่ 1: "Smart Vision Journal" (สมุดบันทึกอัจฉริยะ)

- **ฟีเจอร์หลัก:** แอปจดบันทึกที่ผู้ใช้สามารถถ่ายรูปเอกสาร กระดานดำ หรือสไลด์เรียน แล้วแอปใช้ *Google ML Kit* ดึงข้อความออกมา จากนั้นใช้ *LLM API* ช่วยสรุปประเด็นสำคัญให้อัตโนมัติ ข้อมูลทั้งหมดบันทึกเก็บไว้ใน *Local Database* สามารถค้นหาและอ่านย้อนหลังได้แม้ออฟไลน์

ตัวอย่างที่ 2: "AI Receipt & Expense Tracker" (แอปจัดการรายรับรายจ่ายจากใบเสร็จ)

- **ฟีเจอร์หลัก:** ผู้ใช้ถ่ายรูปใบเสร็จรับเงิน ใช้ *Google ML Kit* สแกนราคารวมและชื่อร้านค้า ส่งข้อมูลให้ *LLM API* ช่วยแยกหมวดหมู่ค่าใช้จ่าย (เช่น อาหาร, เดินทาง) อัตโนมัติ จากนั้นบันทึกลง *Local Database* และมีหน้า Dashboard สรุปค่าใช้จ่าย

ตัวอย่างข้อกำหนดของระบบ (System Requirements)

1. สถาปัตยกรรมและโครงสร้างโค้ด (Architecture & Core)

- **Clean Architecture & Repository Pattern:** ต้องแบ่งเลเยอร์ของแอปพลิเคชันอย่างชัดเจน (Domain, Data, Presentation) และใช้ Repository Pattern ในการจัดการแหล่งข้อมูล
- **Dependency Injection:** ใช้แพ็กเกจ `get_it` ในการจัดการ Dependencies ทั้งหมด
- **Functional Programming:** ใช้แพ็กเกจ `dartz` สำหรับจัดการ Error Handling ในระดับ Domain Layer (ใช้ `Either<Failure, Success>`)

2. การจัดการสถานะและการนำทาง (State Management & Navigation)

- **State Management:** เลือกใช้ **BLoC** หรือ **Riverpod** (อย่างใดอย่างหนึ่ง) ในการจัดการสถานะของแอปพลิเคชันทั้งหมด (ไม่อนุญาตให้ใช้แค่ `setState` ในลอจิกหลัก)
- **Navigation:** ต้องใช้แพ็กเกจ `auto_route` สำหรับจัดการเส้นทาง (Routing) ทั้งหมดในแอป รวมถึงการส่งพารามิเตอร์ข้ามหน้า

3. ส่วนติดต่อผู้ใช้และแอนิเมชัน (UI, Forms & Animations)

- **Form & Validation:** มีหน้าจอเพิ่ม/แก้ไขบันทึก (TextFormField) พร้อมระบบ Validate ข้อมูล (เช่น ห้ามว่าง, ความยาวขั้นต่ำ) โดยใช้ `GlobalKey<FormState>`
- **Animations:** * บังคับให้มี **Implicit Animation** (เช่น `AnimatedContainer` หรือ `AnimatedOpacity`) เมื่อมีการเปลี่ยนสถานะ UI
    - บังคับให้มี **Explicit Animation** (เช่น `Hero` widget) เมื่อกดดูรายละเอียดของบันทึกจากหน้ารวม (List) ไปยังหน้าจอรายละเอียด (Detail)

4. ฐานข้อมูลและการรองรับออฟไลน์ (Databases & Offline Support)

แอปพลิเคชันต้องทำงานได้แม้ไม่มีอินเทอร์เน็ต (Offline-first) โดยบังคับใช้เครื่องมือต่อไปนี้:

- **SharedPreferences:** ใช้สำหรับบันทึกการตั้งค่าของผู้ใช้ เช่น สลับธีมสว่าง/มืด (Light/Dark Mode)
- **Relational/Object Database (เลือก 1 อย่าง: SQLite (sqflite) / Isar / Drift):** ใช้เป็นฐานข้อมูลหลัก (Local Database) สำหรับเก็บข้อมูลบันทึก (Notes) ทั้งหมด
- **NoSQL / Key-Value (เลือก 1 อย่าง: Hive):** ใช้สำหรับทำ **Caching Strategy** เก็บผลลัพธ์ชั่วคราวที่ได้จากการเรียก API สรุปข้อความของ AI เพื่อลดการเรียก API ซ้ำซ้อน

5. การเชื่อมต่อเครือข่ายและ AI (Networking, API & AI Integration)

- **Google ML Kit (On-device ML):** มีฟีเจอร์ถ่ายภาพหรือเลือกรูปจากแกลเลอรี แล้วใช้ ML Kit (Text Recognition) ดึงข้อความออกมาใส่ใน Form
- **LLM Call & REST API:**
    - มีปุ่ม "สรุปข้อความด้วย AI" โดยแอปต้องเชื่อมต่อกับ Public LLM API (เช่น Gemini API หรือ OpenAI API)
    - บังคับใช้แพ็กเกจ `dio` ในการทำ HTTP Request และจัดการ Interceptors (เช่น การแนบ API Key หรือพิมพ์ Log)

    - บังคับใช้ `json_serializable` หรือ `freezed` ในการทำ JSON Serialization แปลงข้อมูล Response จาก API ให้เป็น Dart Object

6. การทดสอบซอฟต์แวร์ (Testing)

ผู้สอบจะต้องเขียนโค้ดทดสอบให้ครอบคลุมตามพีระมิดการทดสอบ (Testing Pyramid):

- **Unit Test:** ทดสอบ Business Logic ใน Use Cases และ BLoC/Notifier โดยใช้การ Mocking (เช่น `mocktail` หรือ `mockito`)
- **Widget Test:** ทดสอบหน้าจอ Form กรอกข้อมูล ตรวจสอบว่าระบบ Validation ทำงานถูกต้องเมื่อผู้ใช้กรอกข้อมูลผิด
- **Integration Test:** เขียนสคริปต์ทดสอบ End-to-End ตั้งแต่เปิดแอป -> กรอกข้อมูล -> กดบันทึก -> ข้อมูลแสดงในหน้ารวม

📂 เกณฑ์การส่งมอบงาน (Submission Guidelines)

1. **Source Code:** อัปโหลดลง GitHub Repository (Public) พร้อมไฟล์ `README.md` อธิบายวิธีการรันโปรเจกต์และ Architecture ที่ใช้
2. **Demonstration Video:** คลิปวิดีโอความยาวไม่เกิน 5-10 นาที สาธิตการทำงานของแอปพลิเคชัน โชว์ฟีเจอร์ออฟไลน์ โชว์แอนิเมชัน และรันผล Test ให้ดู
3. **No API Keys in Code:** ห้ามนำ API Key ของ LLM ผูกติดมากับ Source Code เด็ดขาด ต้องจัดการผ่านไฟล์ `.env` หรือตัวแปรสภาพแวดล้อม